

CVX4PY: A modeling framework in Python for Disciplined Convex Programs

Sayantana Sarkar, Arthita Ghosh and Hui Ding

I. INTRODUCTION

Python is a popular, general purpose language that supports convex optimization problems through packages like CVXPY [11], CVXOPT [12], QCML (ECOS) [13,14] and PICOS, but they have some limitations which we hope to mitigate. To do so, we propose to develop a CVX like modeling framework (CVX4PY) for Python. We also formulate a real world problem as a convex optimization program that is solved through the CVX4PY package¹.

A. Limitations of current Python packages for convex problems

Firstly, most packages require that the convex program be defined programmatically, which can be cumbersome, especially for large problems. Secondly, geometric programming (GP) has very little native support in all of the Python packages we have encountered. The user must transform the GP in *log-sum-exp* form manually and only then can solvers solve the problem. Again the user must manually exponentiate the answers to get the desired values. This process is tedious and error prone. Thirdly, we have explored 3 packages in Python, namely QCML (ECOS), CVXPY and PICOS. QCML is non-programmatic, hence easy to use but can only solve SOCPs. CVXPY is fairly general, but does not support complex SDPs. PICOS supports complex SDPs, but is not as popular as CVXPY. Therefore there are multiple packages, each with some pros and cons.

As a case study we try to tackle the multicell coordination beamforming problem. First we reformulate it and boil it down to solving a complex SDP. We compare the currently available packages and find that while it is easiest to define problems in QCML, it can only solve SOCPs, hence cannot solve our complex SDP program. Next we considered CVXPY, which can solve SDPs, but only real valued ones. While it is possible to transform complex SDPs to real SDPs, it is tedious. Finally we consider PICOS, which can solve complex SDPs, but the problem must be defined programmatically. Hence we put forth this real world problem as an example that can be solved more conveniently with our package.

B. Contributions

These are the contributions of the CVX4PY package, which seeks to (at least partially) address the problems described in section I-A:

- With our package CVX4PY, defining the problem becomes very easy, intuitive and non-programmatic.
- CVX code written for MATLAB can directly be used in Python.
- Our package lets users define GPs in their native non-convex form. Also it supports some extensions to GPs (Generalized Geometric Program), which is currently not supported in MATLAB's CVX.
- CVX4PY brings all these packages together in a single framework, using QCML (ECOS) to solve a limited number of SOCPs, CVXPY to solve (transformed) Generalized Geometric Programs (GGP) and PICOS to solve standard form SDPs (real and complex). Our main focus was creating a robust GGP [15] modeling environment, so currently only limited forms of SOCPs and SDPs are supported.
- A small library of examples is provided. Some are simple toy examples, while one involves a real world problem formulated as an SDP as described in our case study.

In our case study, we try to minimize power in the multicell coordination beamforming problem, subject to some minimum quality constraint, which till now has been considered to have perfect channel information and a centralized implementation. Our contribution is to formulate a relaxed problem as a complex SDP that offers the following advantage

- We propose a distributed formulation (using primal decomposition) of the multicell coordination beamforming problem, with imperfect channel modeling.

II. COMPONENTS OF CVX4PY

Figure 1 explains the structure of CVX4PY package schematically. It provides an overarching framework that brings together three different packages for solving convex optimization problems in Python, namely ECOS (for SOCPs), CVXPY (for GGPs) and PICOS (for SDPs). It provides an interface to the user to input CVX (MATLAB) code to our package and solves the optimization problem using the above mentioned solvers by means of automatically generated Python code at the back end.

A. Lexer

The user inputs a CVX code to CVX4PY in the form of a string. The first step in interpreting this string is *tokenization*. A tokenizer splits the string into individual tokens which are named in a way that indicates their meanings. Hence the lexer breaks the input string into tokens which have types and values. For example:

¹https://github.com/ssarkar2/cvx4py_2

Input String: $x + y \leq 5$
 Tokenization: $(ID, 'x') (PLUS, '+') (ID, 'y') (LEQ, '<=') (INT, '5')$
 (1)

B. Parser

The next step in processing the tokenized string (output of the lexer) is parsing. Within a parser we define Backus-Naur Form grammar rules. LR parsing is implemented by shifting grammar symbols onto a stack and looking at the stack and the next input token for patterns that match one of the grammar rules. If the incoming token looks like part of a valid grammar rule, it is “shifted” onto the stack. If the top of the stack contains a valid right-hand-side of a grammar rule, it is “reduced” and the symbols replaced with the symbol on the left-hand-side. Otherwise, a syntax error has occurred. A parse is successful only when the parser reaches a state where the symbol stack is empty and there are no more input tokens. A short, schematic example is provided in Figure 2

C. Canonicalization

In this step we transform the input convex optimization problem into standard form and ensure that all problems adhere to Disciplined Convex Problem (DCP) ruleset. The standard forms for GGP and SOCP are given in equations 2 and 10. Canonicalization of GGPs are discussed in more details in the next section.

D. Code Generation

After expressing the convex problem in canonical form, Python code is generated in the back end. This is achieved by forming appropriate structures in the Parse/Canonicalization step to segment and store the objective function, constraints, variable list and parameter list. These are passed on to the next stage where the code generating module automatically writes an equivalent Python code for the optimization problem provided by the user using the information stored in the structures passed by the parser/canonicalizer modules.

E. Solving

Using one of the three solvers (ECOS/CVXPY/PICOS) the generated code is run and the solution is output to a text file, the contents of which are read back and packaged to be sent back to the user’s program. The values of the variables and the optimal values are among the information sent back to the user.

F. Using CVX4PY

Users first need to import the cvx4py package. Then there are two ways of entering the CVX string to CVX4PY. The first is that the user a CVX string in the Python code, and the second is that the user enters a filename that contains a CVX code. The user can define constants in the Python code and

CVX4PY line	Comment
from cvx4py import cvx4py	import
a = 1	define constants
string = ““““cvx_begin	CVX string
variable x	
maximize x	
x <= a	
cvx_end””””	
p = cvx4py(string, 0, locals())	create CVX4PY object
soln = p.solveProblem();	Call Solver

TABLE I: Anatomy of a CVX4PY program

refer to those in the CVX string. After the problem is defined, a single *solveProblem()* function is called to get the solution. Table I shows how CVX4PY can be used. Also the appendix contains multiple examples.

III. DISCIPLINED CONVEX PROGRAMMING IN CVX4PY

The Disciplined Convex Programming (DCP) ruleset is a set of sufficient, but not necessary, conditions for convexity. CVX4PY requires that problems are defined by following the DCP ruleset. In this section we examine the rules governing a GGP formulation in greater details and briefly touch upon how SOCPs and SDPs are handled.

A. Generalized Geometric Program

A basic GGP is defined in eq. 2

$$\begin{aligned}
 &\underset{x \in \mathbb{R}_{++}^n}{\text{minimize}} && f_0(x) \\
 &\text{subject to} && f_i(x) \leq 1, \quad i = 1, \dots, m \\
 &&& g_i(x) = 1, \quad i = 1, \dots, p.
 \end{aligned} \tag{2}$$

where $f_0, \dots, f_m$ are generalized posynomials and $g_i, \dots, g_p$ are monomials. A generalized posynomial can be constructed in terms of posynomials p , generalized posynomials h , and real numbers r as per the rules in eq. 3

$$h = \max(h_1, \dots, h_k), \quad h = h_1^r, \quad h = p \tag{3}$$

In CVX4PY GGPs are formed using the following basic programmatic components:

Variables: Variables are the basic building blocks and are defined in CVX format. For example, they are created as follows: *variable x* or *variables x y(2) z(5)*. They can be multidimensional and are indexable by parentheses as follows: $x(2)$

Monomial: Monomials are composed of the product of a positive number and variables raised to real numbers. A separate class is written for monomials, which stores the coefficient and a dictionary relating the variable name to its exponent. The Monomial class supports various operations like adding new terms, product, division, exponentiation etc.

Constants: Constant numbers are stored as objects of the Monomial class that only have a coefficient and no variables.

Posynomial: Posynomials are expressed as sums of monomials. The data of the Posynomial class is implemented as a list of Monomial class objects. The Posynomial class supports operations like product (with monomials and posynomials),

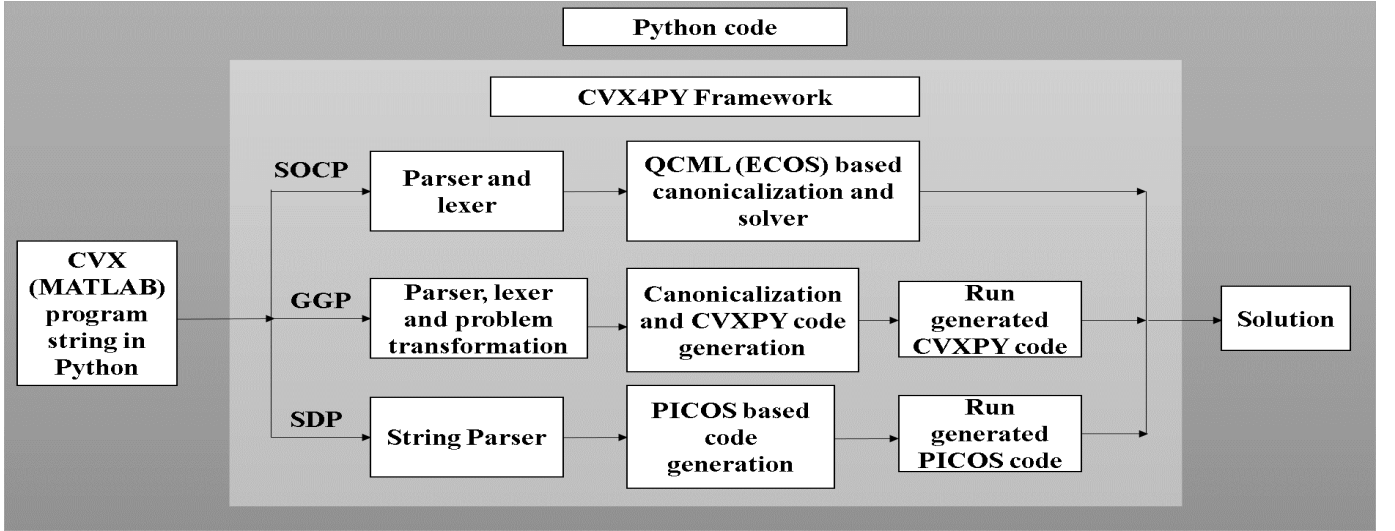


Fig. 1: The CVX4PY Framework

division by monomial, exponentiation by integers, addition to posynomials etc.

GP Constraints: Denoting monomials by m_i and posynomials by p_j , the following constraints are allowed, which are canonicalized to reduce to standard forms.

$$\begin{aligned}
 m_1 = m_2 &\rightarrow m_1/m_2 = 1 \\
 m_1 \leq m_2 &\rightarrow m_1/m_2 \leq 1 \\
 m_1 \geq m_2 &\rightarrow m_2/m_1 \leq 1 \\
 p_1 \leq m_1 &\rightarrow p_1/m_1 \leq 1 \\
 m_1 \geq p_1 &\rightarrow p_1/m_1 \leq 1
 \end{aligned} \quad (4)$$

After the canonicalization, the constraints are converted to convex forms by the usual replacement of an exponentiated variable. Thus we convert the GP constraints to convex constraints as given in eq. 5

$$\begin{aligned}
 ax_1^{q_1} \dots x_n^{q_n} = 1 &\rightarrow \log(a) + \sum_i q_i \bar{x}_i = 0 \\
 ax_1^{q_1} \dots x_n^{q_n} \leq 1 &\rightarrow \log(a) + \sum_i q_i \bar{x}_i \leq 0 \\
 \sum_j a_j x_1^{q_1} \dots x_n^{q_n} \leq 1 &\rightarrow \log(\sum_j e^{\log(a_j) + \sum_i q_i \bar{x}_i}) \leq 0
 \end{aligned} \quad (5)$$

Generalized Posynomial: These are constructed based on the rules given in eq. 3

GGP constraint: Denoting m as monomials, h as generalized posynomials, t as a new variable and r as real numbers we use the rules in eq. 6 to reduce GGP constraints to GP constraints, which can be canonicalized normally as described in eq. 4 and eq. 5.

$$\begin{aligned}
 h^r \leq m &\rightarrow t^r \leq m, \quad h \leq t \\
 \max(h_1, \dots, h_k) \leq m &\rightarrow t \leq m, \quad h_1 \leq t, \dots, h_k \leq t
 \end{aligned} \quad (6)$$

Objective: Letting m denote a monomial and h denote a generalized posynomial, the following objectives are possible: *min.* h , *max.* m or a problem without any objective, which is treated as a feasibility problem.

GGP program: It is composed of variable declarations, at most one objective and GP or GGP constraints

It is important to note here that CVX4PY allows the user most of the flexibility that CVX offers in stating the problem. In fact, the GGP constraints involving max are not

CVX line	feature
$x(3)$	indexing
%comment	comment
$3 * x^3 * y/z$	monomial
$x/(y^{0.4}) == z$	Monomial equality
$x(1) * (z^{0.4}) < z$	Monomial inequality
$y^{0.5} > z$	Monomial inequality
$(x * y) + 5 * y * (z^{0.2})$	posynomial
$x/(z^2) + y * (z^{0.2}) < x^{1.2}$	posynomial inequality
$x^{1.2} > x/y + y * (z^{0.2})$	posynomial inequality
$(x * z + (z * (y^{0.5}))^{0.5}) <= 10$	posynomial
$\max(x, x + z) <= 100$	posynomial
$\max(x, (x + 2 * z)^{0.4}) <= 1$	posynomial
$d1 : 2 * (h * w + h * d) <= 3$	dual variable

TABLE II: Supported features for GGPs

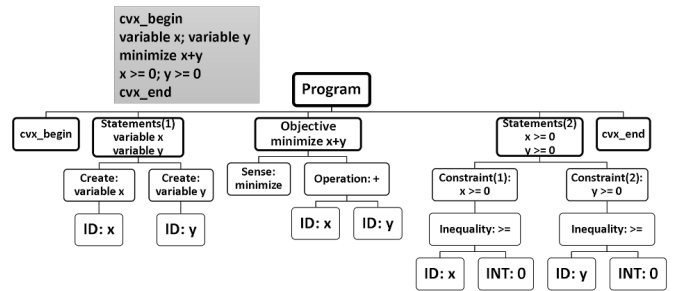


Fig. 2: Parse Tree for a Convex Program

yet supported by CVX, but is supported in CVX4PY. Table II shows a summary of supported features in GGP. Please check appendix for more details and examples.

B. Second Order Conic Program

CVX4PY also supports limited types of SOCPs (eq. 10) including Linear Programs (LP) (eq. 7), some Quadratic Programs (QP) (eq. 8) and Robust LP (eq. 9). The rules for parsing SOCPs are a little different, but of a similar flavor, so we shall not explore them here due to space constraints.

Statement	CVX line	feature
$c^T x$	c^*x	dot product
$Gx \leq h$	$G^*x \leq h$	Linear system of inequalities
$Ax = b$	$A^*x = b$	Linear system of equalities
$0 \leq x \leq 1$	$0 \leq x \leq 1$	chained constraint
$\ x\ _2$	$\text{norm}(x)$	2-norm of a vector
$\sum_i x_i$	$\text{sum}(x)$	sum of a vector

TABLE III: Supported features for SOCPs

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{minimize}} && c^T x \\ & \text{subject to} && Gx \leq h \\ & && Ax = b \end{aligned} \quad (7)$$

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \quad \|Ax - b\|_2^2 \quad (8)$$

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{minimize}} && c^T x \\ & \text{subject to} && a_i^T x + \|P_i^T x\|_2 \leq b_i \quad i = 1 \dots m \end{aligned} \quad (9)$$

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{minimize}} && f^T x \\ & \text{subject to} && Fx = g \\ & && \|A_i x + b_i\|_2 \leq c_i^T x + d_i \quad i = 1 \dots m \end{aligned} \quad (10)$$

Some of the features of SOCPs supported are summarized in table III

C. Semi Definite Program

SDPs (eq. 11) are supported in their standard forms. CVX4PY can handle both real and complex SDPs

$$\begin{aligned} & \underset{x \in \mathbb{S}_n^q}{\text{minimize}} && \text{tr}(CX) \\ & \text{subject to} && \text{tr}(A_i X) = b_i \quad i = 1 \dots p \\ & && X \succeq 0 \end{aligned} \quad (11)$$

IV. CASE STUDY: DISTRIBUTED ROBUST BEAMFORMING

A. Introduction

Although multicell coordination beamforming has been proposed for a long time, the complexity of implementation still makes it hard to realize in practical system due to the need of a central processing unit (CPU) or full cooperation amongst the base stations (BS) associated with large data exchange and signaling overhead. Decentralized beamforming design are considered in [1,2]. However, both assume that the *perfect* Channel State Information (CSI) is available, which is not the case in reality. Motivated by this, the paradigm of robust optimization has been recently considered [3,4]. However, they still can not be implemented in a distributed fashion.

Considering this, we propose a new *distributed robust* downlink beamforming design for a multiuser multicell network. We first reformulate the centralized problem to apply the primal decomposition. Then, using the semidefinite relaxation (SDR) and S-procedure techniques, we convert the QoS constraints with CSI to linear matrix inequalities (LMI), resulting a semidefinite programming (SDP) problem which can be efficiently solved by the existing algorithm.

B. System Model

We consider a multicell wireless system with N_c cells, each cell consists of one base station (BS), which has N_t antennas and K single antenna users. Let $s_{i,k} \in \mathbb{R}, k = 1, \dots, K$ denote the information stream from BS i to its designated users with $E[|s_{i,k}|^2] = 1$ and $\mathbf{w}_{i,k}, k = 1, \dots, K$ are the corresponding beamforming vectors, then the signal received by the user k in cell i can be expressed as:

$$y_{i,k} = \underbrace{\mathbf{h}_{i,i,k}^H \mathbf{w}_{i,k}}_{\text{channel beamformer}} \underbrace{s_{i,k}}_{\text{signal}} + \sum_{l \neq k} \mathbf{h}_{i,i,k}^H \mathbf{w}_{i,l} s_{i,l} + \sum_{j \neq i} \mathbf{h}_{j,i,k}^H \sum_{l=1}^K \mathbf{w}_{j,l} s_{j,l} + \underbrace{z_{i,k}}_{\text{noise}} \quad (12)$$

where H denotes Hermitian transpose. $\mathbf{h}_{i,j,k} \in \mathbb{C}^{N_t \times 1}$ is the downlink channel from BS i to the user k in cell j , which is assumed to be quasi-static flat fading. $z_{i,k}$ is the complex additive white Gaussian noise (AWGN) with zero mean and variance of $\sigma_{i,k}^2$.

From (12), the signal-to-noise-plus-interference ratio (SINR) of the user k in cell i is:

$$\Gamma_{i,k}(\mathbf{h}) = \frac{|\mathbf{h}_{i,i,k}^H \mathbf{w}_{i,k}|^2}{\sum_{l \neq k} |\mathbf{h}_{i,i,k}^H \mathbf{w}_{i,l}|^2 + \sum_{j \neq i} \sum_{l=1}^K |\mathbf{h}_{j,i,k}^H \mathbf{w}_{j,l}|^2 + \sigma_{i,k}^2} \quad (13)$$

Considering the channel estimation errors, we propose to express $\mathbf{h}_{i,j,k}$ as:

$$\mathbf{h}_{i,j,k} = \tilde{\mathbf{h}}_{i,j,k} + \mathbf{e}_{i,j,k} \quad (14)$$

where $\tilde{\mathbf{h}}_{i,j,k}$ is the estimate of $\mathbf{h}_{i,j,k}$ and $\mathbf{e}_{i,j,k}$ is the corresponding error which is confined within an origin-centered hyperspherical region of radius $\epsilon_{i,j,k}$, i.e., $\|\mathbf{e}_{i,j,k}\| \leq \epsilon_{i,j,k}$, where $\|\cdot\|$ denotes the vector Euclidean norm.

In this project, we consider the **Total Transmit Power Minimization (TTPM)** problem as follows:

$$\begin{aligned} & \underset{i,k}{\text{minimize}} && \sum \mathbf{w}_{i,k}^H \mathbf{w}_{i,k} && \text{Minimize Power} \end{aligned} \quad (15)$$

$$\text{subject to} \quad \Gamma_{i,k}(\tilde{\mathbf{h}} + \mathbf{e}) \geq \gamma_{i,k} \quad \text{SINR QoS Constraint}$$

where $\mathbf{w}_{i,k} \in \mathbb{C}^{K \times 1}$ are the optimization variables.

C. Approach

First, we define the intercell interference variables $\beta_{j,i,k}$ as:

$$\sum_{l=1}^K |\mathbf{h}_{j,i,k}^H \mathbf{w}_{j,l}|^2 \leq \beta_{j,i,k}, \quad \forall j \neq i \quad (16)$$

Thus, the problem (4) can be reformulated as:

$$\begin{aligned} & \underset{i,k}{\text{minimize}} && \sum \mathbf{w}_{i,k}^H \mathbf{w}_{i,k} \\ & \text{subject to} && \frac{|\tilde{\mathbf{h}}_{i,i,k} + \mathbf{e}_{i,i,k}|^H \mathbf{w}_{i,k}|^2}{\sum_{l \neq k} |\tilde{\mathbf{h}}_{i,i,k} + \mathbf{e}_{i,i,k}|^H \mathbf{w}_{i,l}|^2 + \sum_{j \neq i} \beta_{j,i,k} + \sigma_{i,k}^2} \geq \gamma_{i,k} \\ & && \sum_{l=1}^K |\tilde{\mathbf{h}}_{j,i,k} + \mathbf{e}_{j,i,k}|^H \mathbf{w}_{j,l}|^2 \leq \beta_{j,i,k}, \quad j \in \{1, \dots, N_c\} \setminus \{i\} \\ & && \forall i = 1, \dots, N_c, k = 1, \dots, K \end{aligned} \quad (17)$$

where $\mathbf{w}_{i,k} \in \mathbb{C}^{K \times 1}$, $\beta_{i,j,k} \in \mathbb{R}$ are the optimization variables. The optimal solution of (17) is achieved when (16) holds with equality, so the problems (15) and (17) are equivalent.

We can see if the intercell interference $\beta_{j,i,k}$ are fixed, then the problem above is separable. Motivated by this, we proposed to split the centralized problem based on Primal Decomposition [5]. Each cell needs to solve the following problems:

$$\begin{aligned} & \text{minimize} \quad \sum_k \mathbf{w}_{i,k}^H \mathbf{w}_{i,k} \\ & \text{subject to} \quad \frac{|(\tilde{\mathbf{h}}_{i,i,k} + \mathbf{e}_{i,i,k})^H \mathbf{w}_{i,k}|^2}{\sum_{l \neq k} |(\tilde{\mathbf{h}}_{i,i,k} + \mathbf{e}_{i,i,k})^H \mathbf{w}_{i,l}|^2 + \sum_{j \neq i}^{N_c} \beta_{j,i,k} + \sigma_{i,k}^2} \geq \gamma_{i,k} \\ & \quad \sum_{l=1}^K |(\tilde{\mathbf{h}}_{i,j,l} + \mathbf{e}_{i,j,l})^H \mathbf{w}_{i,k}|^2 \leq \beta_{i,j,p}, \quad \forall j \neq i, p = 1, \dots, K \\ & \quad \forall i = 1, \dots, N_c, k = 1, \dots, K \end{aligned} \quad (18)$$

where $\mathbf{w}_{i,k} \in \mathbb{C}^{K \times 1}$ is the optimization variable.

However, solving the problem above is still very challenging due to the common channel uncertainty that both in the numerator and the denominator. Thus, we turn to a approximation technique named SDR. By introducing rank relaxation to write $\mathbf{W}_{i,k} = \mathbf{w}_{i,k} \mathbf{w}_{i,k}^H$ and using the S-procedure for the complex case in Appendix A, the infinite SINR constraints can be converted to the following LMIs:

$$\begin{bmatrix} \mathbf{X}_{i,i,k} + \alpha_{i,i,k} \mathbf{I} & \mathbf{X}_{i,i,k} \tilde{\mathbf{h}}_{i,i,k} \\ \tilde{\mathbf{h}}_{i,i,k}^H \mathbf{X}_{i,i,k} & c_{i,i,k} \end{bmatrix} \geq 0 \quad (19)$$

where $\mathbf{X}_{i,i,k} = \frac{1}{\gamma_{i,k}} \mathbf{W}_{i,k} - \sum_{l \neq k} \mathbf{W}_{i,l}$, $c_{i,i,k} = \tilde{\mathbf{h}}_{i,i,k}^H \mathbf{X}_{i,i,k} \tilde{\mathbf{h}}_{i,i,k} - \beta_{j,i,k} - \alpha_{i,i,k} \epsilon_{i,i,k}^2 - \sigma_{i,k}^2$, $\alpha_{i,i,k} \geq 0$.

$$\begin{bmatrix} \mathbf{X}_i + \alpha_{i,j,l} \mathbf{I} & \mathbf{X}_i \tilde{\mathbf{h}}_{i,j,l} \\ \tilde{\mathbf{h}}_{i,j,l}^H \mathbf{X}_i & d_{i,j,l} \end{bmatrix} \geq 0 \quad (20)$$

where $\mathbf{X}_i = -\sum_{k=1}^K \mathbf{W}_{i,k}$, $d_{i,j,l} = \tilde{\mathbf{h}}_{i,j,l}^H \mathbf{X}_i \tilde{\mathbf{h}}_{i,j,l} - \beta_{i,j,l} - \alpha_{i,j,l} \epsilon_{i,j,l}^2$, $\alpha_{i,j,l} \geq 0$.

From the property that $\text{tr}(\mathbf{w}_{i,k}^H \mathbf{w}_{i,k}) = \text{tr}(\mathbf{w}_{i,k} \mathbf{w}_{i,k}^H)$, where ‘ $\text{tr}(\cdot)$ ’ is the trace of a matrix, then the problem (18) can be expressed as:

$$\begin{aligned} & \text{minimize} \quad \sum_k \text{tr}(\mathbf{W}_{i,k}) \\ & \text{subject to} \quad \text{constraints (19), (20)} \\ & \quad \text{rank}(\mathbf{W}_{i,k}) = 1, \mathbf{W}_{i,k} \geq 0 \\ & \quad \alpha_{i,j,k} \geq 0 \\ & \quad \forall i = 1, \dots, N_c, k = 1, \dots, K \end{aligned} \quad (21)$$

where $\mathbf{W}_{i,k} \in \mathbb{C}^{K \times K}$, $\alpha_{i,j,k} \in \mathbb{R}$ are the optimization variables.

If we omit the rank constraint, then (21) becomes a SDP problem which can be efficiently solved by the off-the-shelf optimization package. When $\mathbf{W}_{i,k}$ is directly rank-one, then its principal eigenvector is the optimal solution. Else some approximation techniques should be used to extract the feasible solution from the SDR solution, such as the eigenvector approximation method or the randomization method [6]. The recent result [7] shows that the SDR method is tight when the channel uncertainty bound is small. Thus, we define (21)

without the rank constraint as the *sub-problems*²:

$$\begin{aligned} & \text{minimize} \quad \sum_k \text{tr}(\mathbf{W}_{i,k}) \\ & \text{subject to} \quad \text{constraints (8), (9)} \\ & \quad \mathbf{W}_{i,k} \geq 0 \\ & \quad \forall i = 1, \dots, N_c, k = 1, \dots, K \end{aligned} \quad (22)$$

where $\mathbf{W}_{i,k} \in \mathbb{C}^{K \times K}$, $\alpha_{i,j,k} \in \mathbb{R}$ are the optimization variables. Next, we focus on the *master problem*, which is given by:

$$\begin{aligned} & \text{minimize} \quad \sum_i \phi_i(\beta_i) \\ & \text{s.t.} \quad \beta_i \geq 0 \\ & \quad \forall i = 1, \dots, N_c, k = 1, \dots, K \end{aligned} \quad (23)$$

where ϕ_i is the optimal value of (23) when β_i is given, $\beta_i = [\beta_{j,i,1}, \dots, \beta_{j,i,K}, \beta_{i,j,1}, \dots, \beta_{i,j,K}]$, $j \in \{1, \dots, N_c\} \setminus \{i\}$. The optimization variable is $\beta_i \in \mathbb{R}$.

We can solve this problem by using an iterative method such as the subgradient method [8], i.e.

$$\beta_{i,j,k}(n+1) = [\beta_{i,j,k}(n) - \mu(n)g_{i,j,k}(n)]^+ \quad (24)$$

where n is the iteration index, μ is the step length and $g_{i,j,k}$ is the subgradient of (23) at the point $\beta_{i,j,k}$. $[\cdot]^+$ is the projection onto the nonnegative orthant.

The Lagrange dual problem of (22) is:

$$\begin{aligned} & \text{maximize} \quad \sum_{k=1}^K t_{i,i,k} \left(\sum_{j \neq i}^{N_c} \beta_{j,i,k} + \sigma_{i,k}^2 \right) - \sum_{j \neq i}^{N_c} \sum_{l=1}^K t_{i,j,l} \beta_{i,j,l} \\ & \text{subject to} \quad \mathbf{I} - \left(1 + \frac{1}{\gamma_{i,k}} \right) (\mathbf{Y}_{i,i,k} + \mathbf{y}_{i,k} \tilde{\mathbf{h}}_{i,i,k}^H + \tilde{\mathbf{h}}_{i,i,k} \mathbf{y}_{i,k}^H + t_{i,i,k} \tilde{\mathbf{h}}_{i,i,k} \tilde{\mathbf{h}}_{i,i,k}^H) \\ & \quad + \sum_{j=1}^{N_c} \sum_{l=1}^K (\mathbf{Y}_{i,j,l} + \mathbf{y}_{i,j,l} \tilde{\mathbf{h}}_{i,j,l}^H + \tilde{\mathbf{h}}_{i,j,l} \mathbf{y}_{i,j,l}^H + t_{i,j,l} \tilde{\mathbf{h}}_{i,j,l} \tilde{\mathbf{h}}_{i,j,l}^H) \geq 0 \\ & \quad t_{i,j,k} \epsilon_{i,j,k}^2 - \text{tr}(\mathbf{Y}_{i,j,k}) \geq 0, \quad j \in \{N_c\} \\ & \quad \begin{bmatrix} \mathbf{Y}_{i,j,k} & \mathbf{y}_{i,j,k} \\ \mathbf{y}_{i,j,k}^H & t_{i,j,k} \end{bmatrix} \geq 0, \quad j \in \{N_c\} \\ & \quad \forall i, \{N_c\} = 1, \dots, N_c, k = 1, \dots, K \end{aligned} \quad (25)$$

where $\mathbf{Y}_{i,j,k} \in \mathbb{C}^{K \times K}$, $\mathbf{y}_{i,j,k} \in \mathbb{C}^{N_i \times 1}$, $t_{i,j,k} \in \mathbb{R}$ are the optimization variables.

For the proof, see Appendix B.

From the objective function of (25), we can see the subgradients of (23) at the point $\beta_{i,j,k}$ are given by:

$$g_{i,j,k} = t_{j,j,k} - t_{i,j,k} \quad (26)$$

Each iteration requires solving the subproblems in order to evaluate ϕ_i , $i = 1, \dots, N_c$ and get the corresponding subgradients. The information needed to be exchanged via the backhaul link are only the Lagrange multipliers $t_{i,j,k}$, which are significantly reduced compared with other algorithms. Summarizing the above steps, the primal decomposition based beamformer design is presented in **Algorithm 1**.

²This is the SDP example provided in our package.

Algorithm 1 Distributed Robust Multicell Beamforming

- 1: Initialize the intercell interference $\beta_{i,j,k}(0)$ with some values and set $n=0$.
 - 2: Solve (22) at each BS to get the beamformer $\mathbf{W}_{i,k}$ and the transmit power of BS i .
 - 3: Solve (23) at each BS and exchange the resulting $t_{i,j,k}(n)$ with other BSs via the backhaul network.
 - 4: Every BS updates the local intercell interference $\beta_{i,j,k}(n)$.
 - 5: set $n:=n+1$ and goto step 2 until desired level of convergence.
-

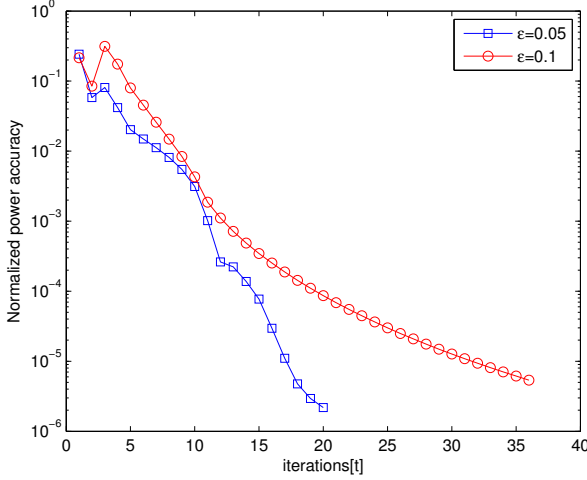


Fig. 3: Convergence behavior of **Algorithm 1**, $N_c=2$, $N_t=4$, $K=2$, $\text{SINR}=10\text{dB}$, ϵ is the error radius.

D. Experiments

In this section, we numerically examine the convergence behavior of our proposed algorithm.

In the simulation, the SINR targets are $\gamma_{i,k}=10$ dB. The channel $\tilde{\mathbf{h}}_{i,j,k} \sim \mathcal{CN}(0, \mathbf{I}_{N_t})$ (a Complex Gaussian) and all the error radii are same, i.e. $\epsilon_{i,j,k} = \epsilon$. We use the CVX [9] to solve the SDP problem. The normalized power accuracy is defined as:

$$r(n) = \frac{|P_{\text{dis}}(n) - P_{\text{cen}}|}{P_{\text{cen}}} \quad (27)$$

where $P_{\text{dis}}(n)$ is the feasible sum of power of **Algorithm 1** at iteration n and P_{cen} is the centralized approach given in [10]. For a single channel realization, we can see from Fig. 1 that the convergence rate is very fast and a relatively good performance can be achieved even after a few iterations. In addition, the convergence of smaller channel estimation error is faster.

V. CONCLUSION

A. Challenges and Limitations

Our goal is to make convex problem modeling in Python (using CVX4PY) as accessible and intuitive as it is in MATLAB (using CVX). This is especially required for GPs and GGP as currently there is no package in Python that supports them natively. However, it is very difficult to map all the functions of CVX into CVX4PY mainly because in CVX, users

can use inbuilt or user defined MATLAB functions to construct expressions. Thus, it is not possible to support all possible MATLAB functions for CVX4PY, and we support only a few important ones in SOCP (for example $\text{sum}()$, $\text{norm}()$, etc). Indexing of variables are not supported for SOCPs, but is supported for GGP. Chained constraints are supported for SOCPs, but not for GGP. Vector-scalar constraints ($x \leq 1$, where x is a vector) are also not supported. Dual variables are supported only in GGP. Also SDPs are limited to the standard form currently.

B. Future Work

Some extensions like chained constraints for GGP and better error handling can be added with relative ease. Some others like supporting more MATLAB functions for SOCPs will take a lot of time, given that MATLAB has a lot of functions. Also we intend to use a proper parser for SDP, and not just a string parser as is currently done.

All in all, initially, we target to make the GGP modeling capabilities of CVX4PY more robust and release worthy, due to a dearth of such a software in Python. Then we can focus on SDPs and SOCPs and make CVX4PY as general purpose as possible.

APPENDIX A S-PROCEDURE

Applying the S-procedure for the complex case [11], we can get the following lemma:

Lemma 1: Given Hermitian matrix $\mathbf{A} \in \mathbb{C}^{n \times n}$, vectors $\mathbf{h}, \mathbf{e} \in \mathbb{C}^{n \times 1}$ and constant c , the two conditions are equivalent:

- 1) for $\forall \mathbf{e}, \mathbf{e}^H \mathbf{e} \leq \epsilon^2$ such that $(\mathbf{h} + \mathbf{e})^H \mathbf{A} (\mathbf{h} + \mathbf{e}) + c \geq 0$;
- 2) there exists $\alpha \geq 0$ such that

$$\begin{bmatrix} \mathbf{A} + \alpha \mathbf{I} & \mathbf{A} \mathbf{h} \\ \mathbf{h}^H \mathbf{A} & \mathbf{h}^H \mathbf{A} \mathbf{h} + c - \alpha \epsilon^2 \end{bmatrix} \geq 0 \quad (28)$$

where $\mathbf{I} \in \mathbb{C}^{n \times n}$ is the identity matrix.

APPENDIX B LAGRANGIAN OF SUBPROBLEM

The Lagrange function of (23) is:

$$\begin{aligned} L = & \sum_{k=1}^K \text{tr}(\mathbf{W}_{i,k}) - \sum_{k=1}^K \text{tr} \left(\begin{bmatrix} \mathbf{Y}_{i,j,k} & \mathbf{y}_{i,j,k} \\ \mathbf{y}_{i,j,k}^H & t_{i,j,k} \end{bmatrix} \begin{bmatrix} \mathbf{X}_{i,j,k} + \alpha_{i,j,k} \mathbf{I} & \mathbf{X}_{i,j,k} \tilde{\mathbf{h}}_{i,j,k} \\ \tilde{\mathbf{h}}_{i,j,k}^H \mathbf{X}_{i,j,k} & c_{i,j,k} \end{bmatrix} \right) \\ & - \sum_{j \neq i} \sum_{l=1}^K \text{tr} \left(\begin{bmatrix} \mathbf{Y}_{i,j,l} & \mathbf{y}_{i,j,l} \\ \mathbf{y}_{i,j,l}^H & t_{i,j,l} \end{bmatrix} \begin{bmatrix} \mathbf{X}_l + \alpha_{i,j,l} \mathbf{I} & \mathbf{X}_l \tilde{\mathbf{h}}_{i,j,l} \\ \tilde{\mathbf{h}}_{i,j,l}^H \mathbf{X}_l & d_{i,j,l} \end{bmatrix} \right) \\ = & \sum_{k=1}^K \text{tr} \left\{ \mathbf{W}_{i,k} \left[\sum_{j=1}^{N_c} \sum_{l=1}^K (\mathbf{Y}_{i,j,l} + \mathbf{y}_{i,j,l} \tilde{\mathbf{h}}_{i,j,l}^H + \tilde{\mathbf{h}}_{i,j,l} \mathbf{y}_{i,j,l}^H + t_{i,j,l} \tilde{\mathbf{h}}_{i,j,l} \tilde{\mathbf{h}}_{i,j,l}^H) \right. \right. \\ & \left. \left. + \mathbf{I} - \left(1 + \frac{1}{\gamma_{i,k}} \right) (\mathbf{Y}_{i,i,k} + \mathbf{y}_{i,i,k} \tilde{\mathbf{h}}_{i,i,k}^H + \tilde{\mathbf{h}}_{i,i,k} \mathbf{y}_{i,i,k}^H + t_{i,i,k} \tilde{\mathbf{h}}_{i,i,k} \tilde{\mathbf{h}}_{i,i,k}^H) \right] \right\} \\ & + \sum_{j=1}^{N_c} \sum_{k=1}^K \alpha_{i,j,k} [t_{i,j,k} \epsilon_{i,j,k}^2 - \text{tr}(\mathbf{Y}_{i,j,k})] + \sum_{k=1}^K t_{i,i,k} \left(\sum_{j \neq i}^{N_c} \beta_{j,i,k} + \sigma_{i,k}^2 \right) \\ & - \sum_{j \neq i} \sum_{l=1}^K t_{i,j,l} \beta_{i,j,l}, \quad \forall i \end{aligned} \quad (29)$$

since $\mathbf{W}_{i,k} \geq 0$, $\alpha_{i,j,k} \geq 0$, the Lagrange dual problem is given as (14).

APPENDIX C

GRAMMAR RULES FOR PARSING GPs

For parsing GP strings, the following grammar rules were used. Capitalized tokens are terminal symbols, whereas non-capitalized ones are non-terminals. Some of these rules have corresponding actions. For example, when rule 12 is encountered, the constraints are canonicalized using eq. 4 and then converted into their convex forms using 5 and stored in a list as equality and inequality constraints. Rule 15 and 16 are used for generalized posynomials and it creates a new monomial variable as described in 6. When rule 3 executes, the objective and sense are noted. Thus taking appropriate actions for each rule the parser collects information about the sense, objective, constraints, declared primal and dual variables. The complete set of actions can be found in the **cvxParserGP.py** module.

When the user inputs a GP string (indicated by `cvx_begin` gp at the start), the GP mode is switched on and the rules contained in **cvxParserGP.py** are used to parse the program string. Similarly, in case of SDP `cvx_begin sdp` indicates that SDP parsing rules need to be used. If nothing is mentioned the default mode is SOCP and the grammar for parsing SOCPs is used. The grammar rules used for parsing SOCPs are similar. The complete set of rules and actions for SOCPs can be found in the **cvxParser.py** module.

1. program : `cvxbegin statements cvxend`
 | `cvxbegin statements objective statements cvxend`
 | `cvxbegin statements objective cvxend`
 | empty
2. empty :
3. objective : `SENSE posy NL` | `SENSE posy NL SUBJECT TO NL`
 | `SENSE mono NL` | `SENSE mono NL SUBJECT TO NL`
4. cvxbegin : `CVX_BEGIN GP SEMICOLON` | `CVX_BEGIN GP COMMA`
 | `CVX_BEGIN GP NL`
5. cvxend : `CVX_END` | `CVX_END NL` | `CVX_END SEMICOLON`
 | `CVX_END COMMA`
6. create : `VARIABLE var` | `VARIABLES varlist`
 | `DUAL VARIABLE ID` | `DUAL VARIABLES idlist`
7. var : `ID LPAREN dimlist RPAREN` | `ID` | `ID LPAREN RPAREN`
8. varlist : `varlist var` | `var`
9. dimlist : `INT` | `ID`
10. statements : `statement NL` | `statement SEMICOLON`
 | `statement COMMA`
 | `statements statement NL`
 | `statements statement SEMICOLON`
 | `statements statement COMMA`
 | `statements statement SEMICOLON NL`
11. statement : `constraint`
 | `create`
 | empty
12. constraint : `mono LOGICALEQUAL mono`
 | `posy LESSTHANEQUAL mono`
 | `mono GREATERTHANEQUAL posy`
 | `mono LESSTHANEQUAL mono`
 | `mono GREATERTHANEQUAL mono`
13. mono : `mono TIMES mono` | `mono DIVIDE mono`
 | `mono POWER FLOAT` | `mono POWER INT` | `var`
 | `INT` | `FLOAT` | `LPAREN mono RPAREN`
14. posy : `mono PLUS mono` | `posy PLUS posy` | `posy PLUS mono`
 | `posy TIMES posy` | `posy TIMES mono`
 | `mono TIMES posy` | `posy DIVIDE mono`
 | `posy POWER INT` | `LPAREN posy RPAREN`
15. monoreplaced : `posy POWER FLOAT`
16. mono : `monoreplaced`
17. list : `list COMMA posy` | `list COMMA mono`
18. maxreplaced : `ATOM LPAREN list RPAREN`
19. mono : `ATOM LPAREN mono RPAREN`
20. posy : `ATOM LPAREN posy RPAREN`
21. mono : `maxreplaced`

APPENDIX D

EXAMPLE OF CONVEX PROGRAMS HANDLED BY CVX4PY

A GGP example	
from cvx4py import cvx4py	
string = """ cvx_begin gp variables x y z dual variable d maximize x*y*z d: x*y + y*z <= 100 (x*z+(z)*(y^0.5))^0.5<=50 max(x, x+z) <= 100 max(x,(x+2*z)^0.4) <= 100 x<=10; y<=20; z<=30 cvx_end"""	Define GP in CVX syntax Dual variable Posynomial constraint <i>Posynomial^{float}</i> (non standard for GP) <i>Max()</i> (non standard for GP) (not supported in cvx) Complex statement (<i>max</i> and <i>Posynomial^{float}</i>) Monomial inequalities
p=cvx4py(string,0,locals()) soln=p.solveProblem(); print soln	

An LP example	
import numpy as np from cvx4py import cvx4py	
a = 14 string = """ cvx_begin variables x y minimize 2*x + 4*y 2*x + y >= a x + y >= 12 x + 3*y >= 18 10 >= x >= 0 y >= 0 cvx_end """	'a' is a constant defined outside the CVX string Chained Constraint
prob = cvx4py(string, 0, locals()) soln = prob.solveProblem(); print soln['x']	

A QP example	
import numpy as np from cvx4py import cvx4py	
A = np.array([[0., 0.], [1., 2.], [2., 1.], [2., 2.]]) b = np.array([0],[9],[7],[9]))	Parameters declared as numpy array
string = """ cvx_begin variables x(2) minimize norm(A*x - b) cvx_end """	Input Program String Unconstrained problem
prob = cvx4py(string, 0, locals()) soln = prob.solveProblem(); print soln['x']	

A Robust LP example	
import numpy as np from cvx4py import cvx4py	
c = np.array([[1],[2]]) A = np.array([[1,-1,0], [-1,0,-1]]); b = np.array([1,0,0]) a1 = np.array([[1],[1]]) ; b1 = 5 P1 = np.array([[1,2],[2,1]])	Parameters of the problem
string = """ cvx_begin variables x(2) maximize c'*x subject to A'*x <= b %a1'*x <= b1 a1'*x + norm(P1*x) <= b1 cvx_end """	Variable Declaration Affine Objective Affine Constraints Comment Norm Constraint
prob = cvx4py(string, 0, locals()) soln = prob.solveProblem(); print soln['x']	

An SDP example	
from cvx4py import cvx4py import cvxopt as cvx	
n = 3 C=cvx.matrix([[1,2,3],[2,9,0],[3,0,7]]) A1=cvx.matrix([[1,0,1],[0,3,7],[1,7,5]]) A2=cvx.matrix([[0,2,8],[2,6,0],[8,0,4]]) b1 = 11; b2 = 19	Parameters for the problem
string = """ cvx_begin sdp variable X(n,n) symmetric minimize(trace(C*X)) subject to trace(A1*X) == b1; trace(A2*X) == b2; X >= 0; cvx_end """	Define SDP Variable Declaration Objective Constraints PSD constraint
p = cvx4py(string,0,locals()) soln = p.solveProblem(); print soln	

REFERENCES

- [1] Y. Huang, G. Zheng, M. Bengtsson, K.-K. Wong, L. Yang and B. Ottersten, "Distributed Multicell Beamforming With Limited Inter-cell Coordination," *IEEE Trans. Signal Process.*, vol. 59, no. 2, pp. 728-738, Feb. 2011.
- [2] H. Dahrouj and W. Yu, "Coordinated Beamforming for the Multicell Multi-Antenna Wireless System," *IEEE Trans. Wireless Commun.*, vol. 9, no. 5, pp. 1748-1759, May 2010.
- [3] A. Tajer, N. Prasad, and X. Wang, "Robust Linear Precoder Design for Multi-Cell Downlink Transmission," *IEEE Trans. Signal Process.*, vol. 59, no. 1, pp. 235 - 251, Jan. 2011.
- [4] D. Ponukumati, F. Gao, M. Bode and X. Liao, "Multicell Downlink Beamforming with Imperfect Channel Knowledge at Both Transceiver Sides," *IEEE Trans. Commun.*, vol. 15, no. 10, pp. 1075-1077, Oct. 2011.

- [5] S. Boyd, L. Xiao, A. Mutapcic and J. Mattingley, "Notes on Decomposition Methods," Feb. 2007. [Online]Available: http://www.stanford.edu/class/ee364b/notes/decomposition_notes.pdf.
- [6] Z.-Q. Luo, W.-K. Ma, A.M.-C. So, Y. Ye and S. Zhang, "Semidefinite Relaxation of Quadratic Optimization Problems," *IEEE Signal Process. Magazine*, vol. 27, no. 3, pp. 20-34, May 2010.
- [7] E. Song, Q. Shi, M. Sanjabi, R. Sun and Z.-Q. Luo, "Robust SINR-constrained MISO downlink beamforming: When is semidefinite programming relaxation tight?," in *Proc. IEEE ICASSP*, Prague, Czech, May 22-27, 2011, pp. 3096-3099.
- [8] S. Boyd and A. Mutapcic, "Subgradient Methods," Apr. 2008. [Online]Available:http://www.stanford.edu/class/ee392o/subgrad_method.pdf.
- [9] M. Grant and S. Boyd, "CVX: Matlab Software for Disciplined Convex Programming, version 1.21," Apr. 2011. [Online]Available: <http://cvxr.com/cvx/download/>.
- [10] C. Shen, K.-Y. Wang, T.-H. Chang, Z. Qiu and C.-Y. Chi, "Worst-Case SINR Constrained Robust Coordinated Beamforming for Multicell Wireless Systems," in *Proc. IEEE Int. Conf. Commun.*, Kyoto, Japan, Jun.5-9, 2011, pp. 1-5.
- [11] Steven Diamond and Stephen Boyd, "CVXPY: A Python-Embedded Modeling Language for Convex Optimization," *Journal of Machine Learning Research*
- [12] M. S. Andersen, J. Dahl, and L. Vandenbergh, "CVXOPT: A Python package for convex optimization"
- [13] E. Chu and S. Boyd, "QCML version 0.2.0: A Python toolbox for Matrix Stuffing"
- [14] A. Domahidi, E. Chu and S. Boyd, "ECOS: An SOCP Solver for Embedded Systems," *2013 European Control Conference (ECC)*
- [15] "A tutorial on geometric programming" *Optim Eng (2007)* 8: 67127 DOI 10.1007/s11081-007-9001-7